

웹시스템개발1

1. 데이터 타입



목차



- 크롬 개발자 도구
- 데이터 타입의 종류
- 데이터 타입에 관한 배경지식
- 변수 선언과 데이터 할당
- 기본형 데이터와 참조형 데이터
- 불변 객체
- undefined와 null

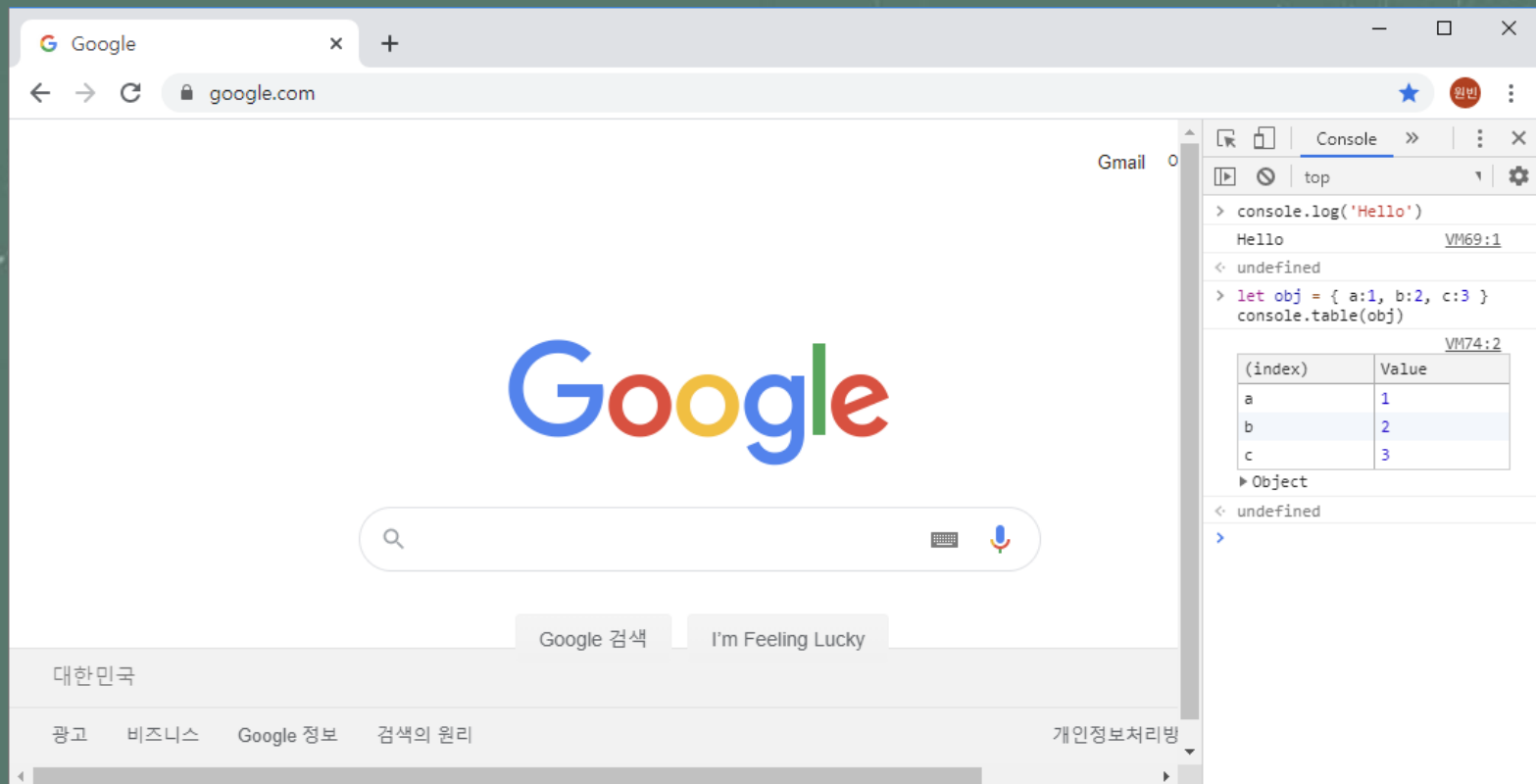


크롬 개발자 도구



단축키

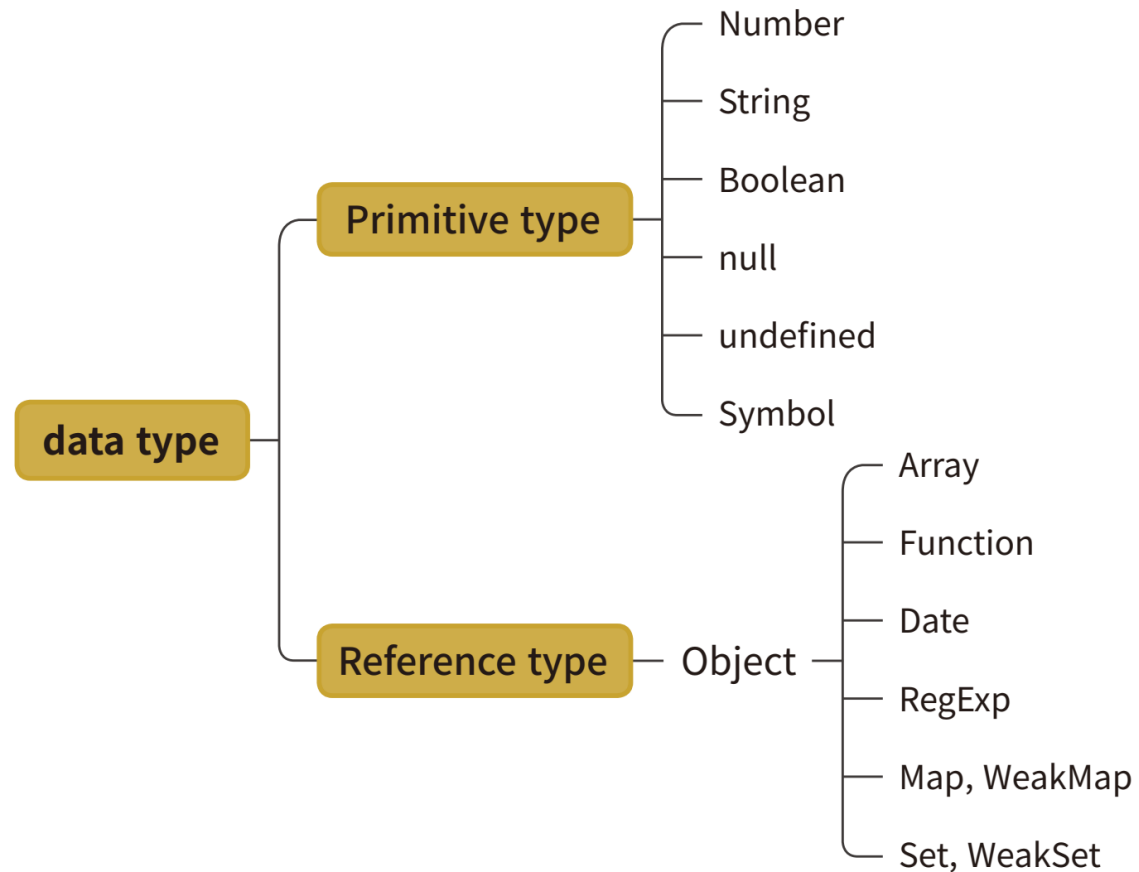
- 콘솔 창 open/close: Ctrl + Shift + J (Cmd + Opt + J)
- 콘솔 창 clear: Ctrl + L



데이터 타입



그림 1-1 데이터 타입의 종류



데이터 타입



- 기본형은 불변성 (immutability)

```
> let a = 15  
a = 10
```

```
< 10
```

```
> a
```

```
< 10
```

데이터 타입에 관한 배경지식



● 정적 언어

- 데이터 타입을 컴파일 시에 결정
- C, C++, C#, Java

● 동적 언어

- 데이터 타입을 런타임에 결정
- Javascript, Ruby, Python

데이터 타입에 관한 배경지식



● Javascript

- 숫자 타입의 경우 정수형인지, 부동소수형인지 구분하지 않고 64비트, 즉 8바이트를 확보

● 각 비트는 고유한 식별자를 가짐

● 바이트는 시작하는 비트의 식별자를 사용

● 모든 데이터는 바이트 단위의 식별자, 더 정확하게 메모리 주소값을 통해 식별

데이터 타입에 관한 배경지식



● 변수

- 변할 수 있는 데이터

● 식별자

- 데이터를 식별하는데 사용하는 이름, 즉 변수명

● 리터럴 (Literal)

- 변수에 할당하는 변하지 않는, 데이터 그 자체

변수 선언과 데이터 할당



- 변할 수 있는 데이터를 만든다. 이 데이터의 식별자는 `a`로 한다.

```
> var a  
← undefined
```

그림 1-3 변수 선언에 대한 메모리 영역의 변화

주소	...	1002	1003	1004	1005	...
데이터			이름: a 값:			

변수 선언과 데이터 할당



1. 변수 영역에서 빈공간(@1003)을 확보한다.
2. 확보한 공간의 식별자를 a로 한다.
3. 데이터 영역의 빈공간(@5004)에 문자열 'abc'를 저장한다.
4. 변수 영역에서 a라는 식별자를 검색한다.
5. 저장한 문자열의 주소(@5004)를 @1003의 공간에 저장한다.

```
01 var a;  
02 a = 'abc';  
03  
04 var a = 'abc';
```

그림 1-4 데이터 할당에 대한 메모리 영역의 변화

변수 영역	주소	...	1002	1003	1004	1005	...
	데이터			이름: a 값: @5004			
데이터 영역	주소	...	5002	5003	5004	5005	...
	데이터				'abc'		

변수 선언과 데이터 할당



```
var a = 'abc';  
a = a + 'def';
```

그림 1-5 문자열 변환에 대한 메모리 영역의 변화

주소	...	1002	1003	1004	1005	...
데이터			이름: a 값: @5005	변경됨		
주소	...	5002	5003	5004	5005	...
데이터				'abc'	'abcdef'	

변수 선언과 데이터 할당



- 500개의 변수를 생성해서 모든 변수에 숫자 5를 할당한다면?
 - 데이터 영역에 총 4000 ($500 * 8$)바이트를 생성하는 것보다 숫자 5를 한 번만 저장하고 해당 주소만 변수 영역에 저장하는게 더 효율적임
 - 즉, 중복된 데이터에 대한 처리 효율 증가

기본형 데이터와 참조형 데이터



- 변수와 상수를 구분하는 성질은 '변경 가능성'
- 변수와 상수를 구분 짓는 변경 가능성의 대상 => '변수 영역'
- 불변성 여부를 구분할 때의 변경 가능성의 대상 => '데이터 영역'

기본형 데이터와 참조형 데이터



```
var a = 'abc';
```

```
a = a + 'def';
```

```
var b = 5;
```

```
var c = 5;
```

```
b = 7;
```

- 기본형 데이터인 문자열과 숫자 값은 다른 값으로 변경할 수 있는 불변 데이터
- 한 번 만들어진 값은 가비지 컬렉터에 의해 메모리가 소멸될 때까지 유지

기본형 데이터와 참조형 데이터



- 기본형 데이터는 모두 불변 값
- 참조형 데이터는?



기본형 데이터와 참조형 데이터



```
var obj1 = {  
  a: 1,  
  b: 'bbb'  
};
```

그림 1-6 참조형 데이터의 할당

변수 영역	주소	1001	1002	1003	1004	...
	데이터		이름: obj1 값: @5001			
데이터 영역	주소	5001	5002	5003	5004	...
	데이터	@7103 ~ ?		1	'bbb'	
객체 @5001의 변수 영역	주소	7103	7104	7105	7106	...
	데이터	이름: a 값: @5003	이름: b 값: @5004			

기본형 데이터와 참조형 데이터



그림 1-7 참조형 데이터의 프로퍼티 재할당

```
var obj1 = {  
  a: 1,  
  b: 'bbb'  
};  
  
obj1.a = 2;
```

변수 영역	주소	1001	1002	1003	1004	1005	...
	데이터		이름: obj1 값: @5001				
데이터 영역	주소	5001	5002	5003	5004	5005	...
	데이터	@7103 ~ ?		1	'bbb'	2	
객체 @5001의 변수 영역	주소	7103	7104	7104	7106	7107	...
	데이터	이름: a 값: @5005	이름: b 값: @5004				

● 객체의 내부의 값만 변경 됨

기본형 데이터와 참조형 데이터



그림 1-8 중첩된 참조형 데이터(객체)의 프로퍼티 할당

```
var obj = {
  x: 3,
  arr: [ 3, 4, 5 ]
};
```

변수 영역	주소	1001	1002	1003	1004	1005	...
	데이터		이름: obj 값: @5001				
데이터 영역	주소	5001	5002	5003	5004	5005	...
	데이터	@7103 ~ ?	3	@8104 ~ ?	4	5	

객체 @5001의 변수 영역

주소	7103	7104	...
데이터	이름: x 값: @5002	이름: arr 값: @5003	

배열 @5003의 변수 영역

주소	8104	8105	8106	...
데이터	이름: 0 값: @5002	이름: 1 값: @5004	이름: 2 값: @5005	

● obj.arr[1]을 검색하고자 한다면?

@1002 -> @5001 -> (@7103~?) -> @7104 -> @5003 -> (@8104~?)
-> @8105 -> @5004 -> 4 반환

기본형 데이터와 참조형 데이터



```
obj.arr = 'str'
```

그림 1-9 중첩된 참조형 데이터(객체)의 프로퍼티 재할당

주소	1001	1002	1003	1004	1005	1006	...
데이터		이름: obj 값: @5001					
주소	5001	5002	5003	5004	5005	5006	...
데이터	@7103 ~ ?	3	@8104 ~ ?	4	5	'str'	

주소	7103	7104	...
데이터	이름: x 값: @5002	이름: arr 값: @5006	

GC 수거 대상

주소	8104	8105	8106	...
데이터	이름: 0 값: @5002	이름: 1 값: @5004	이름: 2 값: @5005	

기본형 데이터와 참조형 데이터



참조 카운트

- 어떤 데이터가 저장된 주소를 참조하는 변수의 개수

가비지 컬렉터

- 참조 카운트가 0이 되면 메모리 수거 대상
- 런타임 환경에 따라 특정 시점이나 메모리 사용량이 포화 상태에 이르면 자동으로 수거 대상들을 수거 (collecting)
- 수거된 메모리는 다시 새로운 값을 할당할 수 있는 빈공간이 됨

그림 1-9 중첩된 참조형 데이터(객체)의 프로퍼티 재할당

주소	1001	1002	1003	1004	1005	1006	...
데이터		이름: obj 값: @5001					
주소	5001	5002	5003	5004	5005	5006	...
데이터	@7103 ~ ?	3	@8104 ~ ?	4	5	'str'	

주소	7103	7104	...
데이터	이름: x 값: @5002	이름: arr 값: @5006	

주소	8104	8105	8106	...
데이터	이름: 0 값: @5002	이름: 1 값: @5004	이름: 2 값: @5005	

GC 수거 대상

기본형 데이터와 참조형 데이터



```
var a = 10;
var b = a;

var obj1 = { c: 10, d: 'ddd' };
var obj2 = obj1;
```

그림 1-10 변수 복사 비교

변수 영역	주소	1001	1002	1003	1004	...
	데이터	이름: a 값: @5001	이름: b 값: @5001	이름: obj1 값: @5002	이름: obj2 값: @5002	
데이터 영역	주소	5001	5002	5003	5004	...
	데이터	10	@7103 ~ ?	'ddd'		
객체 @5002의 변수 영역	주소	7103	7104	...		
	데이터	이름: c 값: @5001	이름: d 값: @5003			

기본형 데이터와 참조형 데이터



```
var a = 10;  
var b = a;  
var obj1 = { c: 10, d: 'ddd' };  
var obj2 = obj1;
```

기본형과 참조형의 가장 큰 차이점

`a !== b`
`obj1 === obj2`

```
b = 15;  
obj2.c = 20;
```

그림 1-11 변수 복사 이후 값을 변경했을 때의 차이점 (1) – 객체의 프로퍼티를 변경했을 때

	값이 달라짐		값이 달라지지 않음			
주소	1001	1002	1003	1004	1005	...
데이터	이름: a 값: @5001	이름: b 값: @5004	이름: obj1 값: @5002	이름: obj2 값: @5002		
주소	5001	5002	5003	5004	5005	...
데이터	10	@7103 ~ ?	'ddd'	15	20	

주소	7103	7104	...
데이터	이름: c 값: @5005	이름: d 값: @5003	

기본형 데이터와 참조형 데이터



- 어떤 데이터 타입이든 변수에 할당하기 위해서는 주소값을 복사해야 하기 때문에, 자바스크립트의 모든 데이터 타입은 참조형이라고 볼 수 있음.
- 다만, 기본형은 주소값을 복사하는 과정이 한 번만 이뤄지고, 참조형은 한 단계를 더 거치게 된다는 차이가 있음.

기본형 데이터와 참조형 데이터



```
var a = 10;  
var b = a;  
var obj1 = { c: 10, d: 'ddd' };  
var obj2 = obj1;
```

```
b = 15;
```

```
obj2 = { c: 20, d: 'ddd' };
```

참조형 데이터가 '가변값'이라고 설명할 때의 '가변'은 참조형 데이터 자체를 변경할 경우가 아니라 그 내부의 프로퍼티를 변경할 때만 성립

변수 복사 이후 값 변경 결과 비교 (2) - 객체 자체를 변경했을 때

	값이 달라짐		값이 달라짐				
	↓		↓				
주소	1001	1002	1003	1004	1005	1006	...
데이터	이름: a 값: @5001	이름: b 값: @5004	이름: obj1 값: @5002	이름: obj2 값: @5006			
주소	5001	5002	5003	5004	5005	5006	...
데이터	10	@7103 ~ ?	'ddd'	15	20	@8204 ~ ?	

주소	7103	7104	...
데이터	이름: c 값: @5001	이름: d 값: @5003	

주소	8204	8205	...
데이터	이름: c 값: @5005	이름: d 값: @5003	

불변 객체



● 객체의 가변성에 따른 문제점

- 정보가 변경되기 전 정보가 변경된 후의 정보의 차이를 가시적으로 보여줘야 하는 등의 기능을 구현하려면?
- 객체의 내부 프로퍼티를 변경할 필요가 있을 때마다 매번 새로운 객체를 만들어 재할당하는 동작으로 객체를 만드는데 필요한 메모리를 절약할 수 있음

```
01 var user = {  
02   name: 'Jaenam',  
03   gender: 'male'  
04 };  
05  
06 var changeName = function (user, newName) {  
07   var newUser = user;  
08   newUser.name = newName;  
09   return newUser;  
10 };  
11  
12 var user2 = changeName(user, 'Jung');  
13  
14 if (user !== user2) {  
15   console.log('유저 정보가 변경되었습니다.');16 }  
17 console.log(user.name, user2.name);    // Jung Jung  
18 console.log(user === user2);          // true
```


불변 객체



```
01 var user = {
02   name: 'Jaenam',
03   gender: 'male'
04 };
05
06 var changeName = function (user, newName) {
07   return {
08     name: newName,
09     gender: user.gender
10   };
11 };
12
13 var user2 = changeName(user, 'Jung');
14
15 if (user !== user2) {
16   console.log('유저 정보가 변경되었습니다.');// 유저 정보가 변경되었습니다.
17 }
18 console.log(user.name, user2.name);           // Jaenam Jung
19 console.log(user === user2);                  // false
```

새로운 객체를 만들어 리턴하고 있으며, 매우 비효율적

불변 객체



- 기존 정보를 복사해서 새로운 객체를 반환하는 함수

```
01 var copyObject = function (target) {  
02     var result = {};  
03     for (var prop in target) {  
04         result[prop] = target[prop];  
05     }  
06     return result;  
07 };
```

불변 객체



```
01 var user = {
02     name: 'Jaenam',
03     gender: 'male'
04 };
05
06 var user2 = copyObject(user);
07 user2.name = 'Jung';
08
09 if (user !== user2) {
10     console.log('유저 정보가 변경되었습니다.');// 유저 정보가 변경되었습니다.
11 }
12 console.log(user.name, user2.name);           // Jaenam Jung
13 console.log(user === user2);                  // false
```

불변 객체



- 모든 개발자들이 user 객체 내부의 프로퍼티를 변경할 때는 무조건 `copyObject` 함수를 사용하기로 합의한다면 user 객체는 불변 객체가 될 수 있음.
- 하지만 모두가 그 규칙을 따르지 않고는 프로퍼티를 변경을 할 수 없게끔 제약을 거는 편이 더 안전함.
- `immutable.js`, `baobab.js` 등의 라이브러리가 인기
 - 자바스크립트 내장 객체가 아닌 라이브러리 자체에서 불변성을 지닌 별도의 데이터 타입과 그에 따른 메소드를 제공

불변 객체



```
01 var user = {
02   name: 'Jaenam',
03   urls: {
04     portfolio: 'http://github.com/abc',
05     blog: 'http://blog.com',
06     facebook: 'http://facebook.com/abc'
07   }
08 };
09 var user2 = copyObject(user);
10
11 user2.name = 'Jung';
12 console.log(user.name === user2.name);           // false
13
14 user.urls.portfolio = 'http://portfolio.com';
15 console.log(user.urls.portfolio === user2.urls.portfolio); // true
16
17 user2.urls.blog = '';
18 console.log(user.urls.blog === user2.urls.blog);  // true
```

● 중첩된 객체
에 대한 얕은
복사

불변 객체



● 중첩된 객체에 대한 깊은 복사 (deep copy)

```
01 var user2 = copyObject(user);  
02 user2.urls = copyObject(user.urls);  
03  
04 user.urls.portfolio = 'http://portfolio.com';  
05 console.log(user.urls.portfolio === user2.urls.portfolio); // false  
06  
07 user2.urls.blog = '';  
08 console.log(user.urls.blog === user2.urls.blog);           // false
```


불변 객체



● 객체의 깊은 복사를 수행하는 함수

```
01 var copyObjectDeep = function(target) {  
02     var result = {};  
03     if (typeof target === 'object' && target !== null) {  
04         for (var prop in target) {  
05             result[prop] = copyObjectDeep(target[prop]);  
06         }  
07     } else {  
08         result = target;  
09     }  
10     return result;  
11 };
```

> typeof(null)

< "object"

불변 객체



예제 1-17 깊은 복사 결과 확인

```
01 var obj = {  
02     a: 1,  
03     b: {  
04         c: null,  
05         d: [1, 2]  
06     }  
07 };  
08 var obj2 = copyObjectDeep(obj);  
09  
10 obj2.a = 3;  
11 obj2.b.c = 4;  
12 obj.b.d[1] = 3;  
13  
14 console.log(obj);    // { a: 1, b: { c: null, d: [1, 3] } }  
15 console.log(obj2);   // { a: 3, b: { c: 4, d: {0:1, 1:2} } }
```

불변 객체



예제 1-18 JSON을 활용한 간단한 깊은 복사

```
01 var copyObjectViaJSON = function (target) {  
02     return JSON.parse(JSON.stringify(target));  
03 };  
04 var obj = {  
05     a: 1,  
06     b: {  
07         c: null,  
08         d: [1, 2],  
09         func1: function () { console.log(3); }  
10     },  
11     func2: function () { console.log(4); }  
12 };  
13 var obj2 = copyObjectViaJSON(obj);  
14  
15 obj2.a = 3;  
16 obj2.b.c = 4;  
17 obj.b.d[1] = 3;  
18  
19 console.log(obj); // { a: 1, b: { c: null, d: [1, 3], func1: f(), func2: f() }  
20 console.log(obj2); // { a: 3, b: { c: 4, d: [1, 2] } }
```

● httpRequest로 받은 데이터를 저장한 객체를 복사할 때 등 순수한 정보만 다를 때는 좋은 방법

undefined와 null



● '없음'을 나타내는 두 가지 방법

● undefined

- 값을 대입하지 않은 변수, 즉 데이터 영역의 메모리 주소를 지정하지 않은 식별자에 접근할 때 undefined를 반환
- 'var a' 구문에서 변수 a에 자동으로 undefined가 할당되는 것이 아닌 변수 a에 접근하고자 할 때 자바스크립트 엔진이 undefined를 반환

undefined와 null



예제 1-19 자동으로 undefined를 부여하는 경우

```
01 var a;  
02 console.log(a);      // (1) undefined. 값을 대입하지 않은 변수에 접근  
03  
04 var obj = { a: 1 };  
05 console.log(obj.a);  // 1  
06 console.log(obj.b);  // (2) 존재하지 않는 프로퍼티에 접근  
07 console.log(b);      // c.f) ReferenceError: b is not defined  
08  
09 var func = function() { };  
10 var c = func();      // (3) 반환(return) 값이 없으면 undefined를 반환한 것으로 간주.  
11 console.log(c);      // undefined
```

undefined와 null



- 배열에서 '비어있는 요소'와 'undefined'를 할당한 요소'는 출력 결과가 다름

예제 1-20 undefined와 배열

```
01 var arr1 = [];  
02 arr1.length = 3;  
03 console.log(arr1);           // [empty x 3]  
04  
05 var arr2 = new Array(3);  
06 console.log(arr2);           // [empty x 3]  
07  
08 var arr3 = [undefined, undefined, undefined];  
09 console.log(arr3);           // [undefined, undefined, undefined]
```


undefined와 null



- '비어있는 요소'는 순회와 관련된 메소드들의 순회 대상에서 제외
 - arr1의 'undefined'는 하나의 값이기에 프로퍼티나 배열의 요소는 고유한 키값이 존재하게 됨, 순회의 대상
 - arr2의 'undefined'는 자바스크립트 엔진이 반환하는 'undefined'이며 배열의 키 값 (인덱스) 자체가 존재하지 않음
- 배열은 length 프로퍼티의 크기 만큼 빈 공간을 확보하는 것이 아니고 객체와 마찬가지로 특정 인덱스에 값을 지정할 때 빈 공간을 확보

```
01 var arr1 = [undefined, 1];
02 var arr2 = [];
03 arr2[1] = 1;
04
05 arr1.forEach(function (v, i) { console.log(v, i); }); // undefined 0 / 1 1
06 arr2.forEach(function (v, i) { console.log(v, i); }); // 1 1
```

undefined와 null



예제 1-21 빈 요소와 배열의 순회

```
01 var arr1 = [undefined, 1];
02 var arr2 = [];
03 arr2[1] = 1;
04
05 arr1.forEach(function (v, i) { console.log(v, i); }); // undefined 0 / 1 1
06 arr2.forEach(function (v, i) { console.log(v, i); }); // 1 1
07
08 arr1.map(function (v, i) { return v + i; }); // [NaN, 2]
09 arr2.map(function (v, i) { return v + i; }); // [empty, 2]
10
11 arr1.filter(function (v) { return !v; }); // [undefined]
12 arr2.filter(function (v) { return !v; }); // []
13
14 arr1.reduce(function (p, c, i) { return p + c + i; }, ''); // undefined011
15 arr2.reduce(function (p, c, i) { return p + c + i; }, ''); // 11
```

undefined와 null



예제 1-22 undefined와 null의 비교

```
01 var n = null;
02 console.log(typeof n);           // object
03
04 console.log(n == undefined);     // true
05 console.log(n == null);          // true
07 console.log(n === undefined);    // false
08 console.log(n === null);         // true
```



Thank you!

